



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной
и дополненной реальности»

Практическая работа № 5

Студент группы *ИКБО-50-23, Павлов Н.С.*

(подпись)

Старший
преподаватель *Горбатов Г.В.*

(подпись)

Отчет представлен «___» _____ 2026 г.

Москва 2026 г.

1. В ходе выполнения практической работы была реализована многопользовательская система с использованием Mirror Networking – внешней сетевой библиотеки для Unity.

Было создано peer-to-peer подключение между участниками: один пользователь выступает в роли Host (сервер + клиент), остальные подключаются как Client

Для реализации использовались:

- Mirror Networking (внешняя система)
- KcpTransport (надёжный UDP транспорт)
- NetworkManager для управления сетевой сессией
- NetworkIdentity и NetworkTransform для синхронизации объектов

Реализован пользовательский интерфейс с:

- Выбором режима: Desktop / XR
- Выбором роли: Host / Client
- Полем для ввода IP-адреса сервера

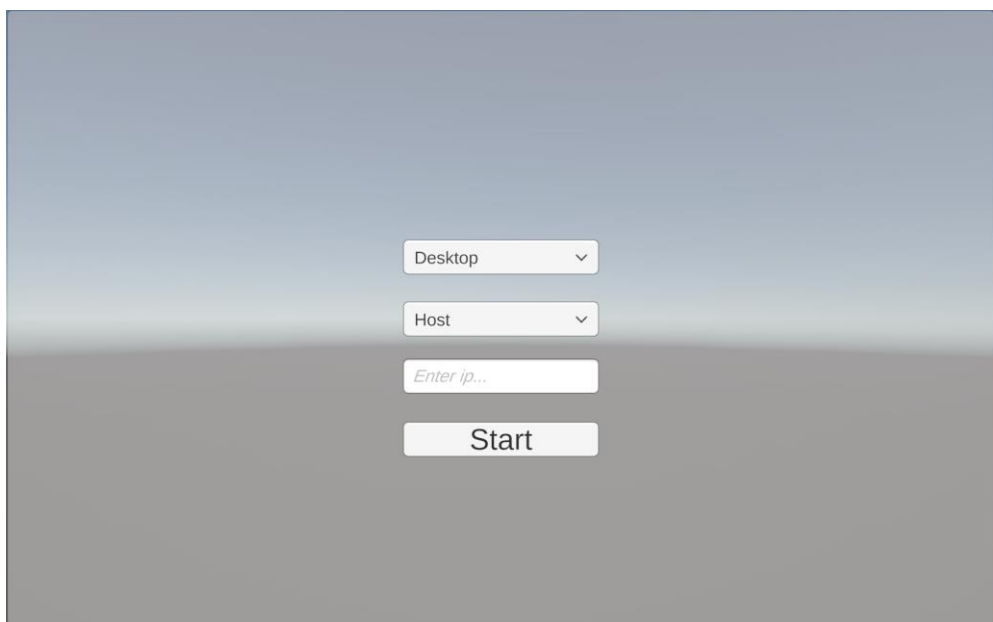


Рисунок 1 — Интерфейс подключения к игровой сессии

Листинг 1 - Скрипт создания сетевого подключения

```
using Mirror;  
using UnityEngine;  
  
public class GameNetworkManager : MonoBehaviour
```

```

{
    public static GameNetworkManager Instance;

    private void Awake() => Instance = this;

    private void Start()
    {
        int mode = PlayerPrefs.GetInt("GameMode");
        bool isHost = PlayerPrefs.GetInt("IsHost") == 1;
        string ip = PlayerPrefs.GetString("ServerIP");

        if (isHost)
        {
            Debug.Log("[NETWORK] Starting as HOST...");
            NetworkManager.singleton.StartHost();
        }
        else
        {
            Debug.Log($"[NETWORK] Connecting to {ip} as CLIENT...");
            NetworkManager.singleton.networkAddress = ip;
            NetworkManager.singleton.StartClient();
        }
    }
}

```

2. Для передачи пользовательских данных была реализована система кастомных сетевых пакетов PTS (Player Telemetry State) с использованием встроенной системы сообщений Mirror.

Листинг 2 - Структура пакета PTS

```

public struct PTS_Packet : NetworkMessage
{
    public uint netId;           // ID объекта в сети
    public Vector3 position;     // Позиция игрока
    public Quaternion rotation;  // Поворот
    public Color syncColor;      // Цвет объекта
    public float timestamp;      // Время отправки
}

```

Каждому игроку автоматически назначается:

- Уникальный случайный цвет капсулы
- Сетевой идентификатор (NetId)
- Возможность перемещения по платформе

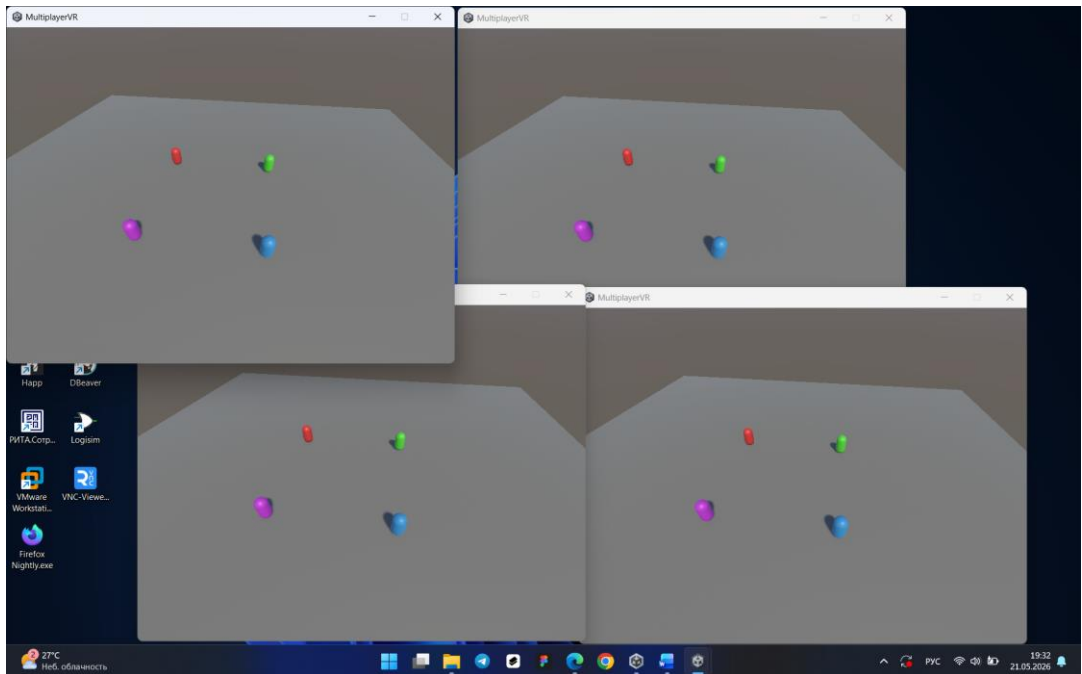


Рисунок 2 — Подключение нескольких игроков к серверу

Листинг 3 - Скрипт обработки PTS пакетов

```
using Mirror;
using UnityEngine;

public struct PTS_Packet : NetworkMessage
{
    public uint netId;
    public Vector3 position;
    public Quaternion rotation;
    public Color syncColor;
    public float timestamp;
}

public class PTSNetworkHandler : NetworkBehaviour
{
    public static PTSNetworkHandler Instance;

    private void Awake() => Instance = this;

    public override void OnStartServer()
    {
        NetworkServer.RegisterHandler<PTS_Packet>(OnReceivePTSPacket);
        Debug.Log("[PTS] Server registered packet handler");
    }

    public override void OnStartClient()
    {
        NetworkClient.RegisterHandler<PTS_Packet>(OnReceivePTSPacket);
        Debug.Log("[PTS] Client registered packet handler");
    }
}
```

```

public void SendPTSPacket(uint id, Vector3 pos, Quaternion rot, Color col)
{
    if (!NetworkServer.active) return;

    PTS_Packet packet = new PTS_Packet
    {
        netId = id,
        position = pos,
        rotation = rot,
        syncColor = col,
        timestamp = Time.time
    };

    NetworkServer.SendToAll(packet);
}

private void OnReceivePTSPacket(NetworkConnection conn, PTS_Packet msg)
{
    if (msg.netId == 0)
    {
        Debug.LogWarning("[PTS] Invalid packet received");
        return;
    }

    if (Time.frameCount % 60 == 0)
    {
        Debug.Log($"[PTS PROCESSED] ID:{msg.netId} | " +
            $"Pos:{msg.position:F2} | " +
            $"Color:{msg.syncColor} | " +
            $"Time:{msg.timestamp:F2}");
    }
}
}

```

Листинг 4 - Скрипт управления игроком

```

using Mirror;
using UnityEngine;

public class PlayerController : NetworkBehaviour
{
    [SerializeField] private float moveSpeed = 5f;
    private Renderer _renderer;
    private Color _myColor;

    // Синхронизация цвета через SyncVar (автоматическая сеть)
    [SyncVar(hook = nameof(OnColorChanged))]
    private Color networkColor;

    private void Awake()
    {

```

```

        _renderer = GetComponent<Renderer>();
    }

    public override void OnStartClient()
    {
        // Генерируем случайный цвет только для локального игрока
        if (isLocalPlayer)
        {
            _myColor = Random.ColorHSV(0f, 1f, 0.8f, 1f, 0.8f, 1f);
            CmdSetInitialColor(_myColor);

            // Настройка камеры зависит от режима
            SetupCameraForMode();
        }
    }

    [Command]
    private void CmdSetInitialColor(Color color)
    {
        networkColor = color;
    }

    private void OnColorChanged(Color oldColor, Color newColor)
    {
        if (_renderer != null)
            _renderer.material.color = newColor;
    }

    private void Update()
    {
        if (!isLocalPlayer) return;

        HandleMovement();
        SendPTSDDataPeriodically();
    }

    private void HandleMovement()
    {
        // Простое движение для Desktop и XR
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");

        Vector3 move = new Vector3(h, 0, v) * moveSpeed * Time.deltaTime;
        transform.Translate(move);
    }

    private void SendPTSDDataPeriodically()
    {
        // Отправляем пользовательский пакет PTS каждые 0.5 секунды
        if (Time.frameCount % 30 == 0)
        {

```

```

        if (PTSTNetworkHandler.Instance != null)
        {
            PTSTNetworkHandler.Instance.SendPTSPacket(
                netId,
                transform.position,
                networkColor
            );
        }
    }

private void SetupCameraForMode()
{
    int mode = PlayerPrefs.GetInt("GameMode", 0);

    if (mode == 0) // Desktop
    {
        // Включаем статичную камеру сверху
        Camera mainCam = Camera.main;
        if (mainCam != null)
        {
            mainCam.transform.position = new Vector3(0, 15, 0);
            mainCam.transform.rotation = Quaternion.Euler(90, 0, 0);
            mainCam.orthographic = true;
            mainCam.orthographicSize = 10;
        }
    }
    else // XR
    {
        // В XR камера управляется шлемом, ничего делать не нужно
    }
}
}

```

3. В проект была интегрирована XR-подсистема с использованием:

- XR Interaction Toolkit (версия 2.x/3.x)
- OpenXR Plugin для кросс-платформенной совместимости
- Unity Input System для унифицированного ввода

Реализованна функциональность переключения между Desktop и XR режимами, XR Origin с контроллерами для VR, система перемещения (Teleportation и Continuous Move)

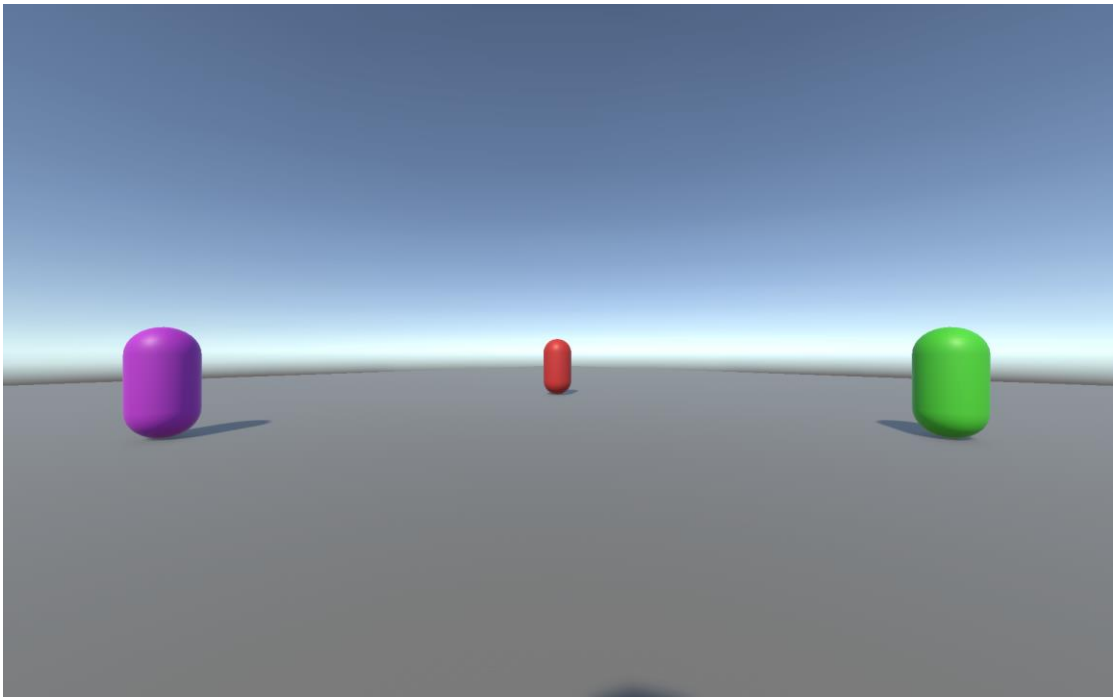


Рисунок 3 — Работа проекта в XR режиме

Листинг 5 - Скрипт переключения режимов

```
using UnityEngine;
using UnityEngine.XR.Management;

public class XRSetup : MonoBehaviour
{
    public GameObject xrOrigin;
    public Camera desktopCamera;

    private void Start()
    {
        int mode = PlayerPrefs.GetInt("GameMode", 0);

        if (mode == 1) // XR Mode
        {
            EnableXR();
            if (desktopCamera) desktopCamera.enabled = false;
            if (xrOrigin) xrOrigin.SetActive(true);
            Debug.Log("[XR] XR Mode enabled");
        }
        else // Desktop Mode
        {
            DisableXR();
            if (desktopCamera) desktopCamera.enabled = true;
            if (xrOrigin) xrOrigin.SetActive(false);
            Debug.Log("[XR] Desktop Mode enabled");
        }
    }
}
```



```
private void EnableXR()
{
    var settings = XRGeneralSettingsPerBuildTarget
        .XRGeneralSettingsForBuildTarget(BuildTarget.StandaloneWindows);

    if (settings != null)
    {
        settings.Manager.InitializeLoaderSync();
        settings.Manager.StartSubsystems();
    }
}

private void DisableXR()
{
    var settings = XRGeneralSettingsPerBuildTarget
        .XRGeneralSettingsForBuildTarget(BuildTarget.StandaloneWindows);

    if (settings != null)
    {
        settings.Manager.StopSubsystems();
        settings.Manager.DeinitializeLoader();
    }
}
}
```